Building a Root Cause Analytics Engine with Prometheus, Grafana, and MCP



El Muhammad Cholid H. · Founder, DevOps Focus Group

Kubernetes Community Days Indonesia 2026

When an alert fires at 3am, the clock starts ticking

Alerts tell you users are hurting, but never why

The hardest part of on-call isn't typing the fix:

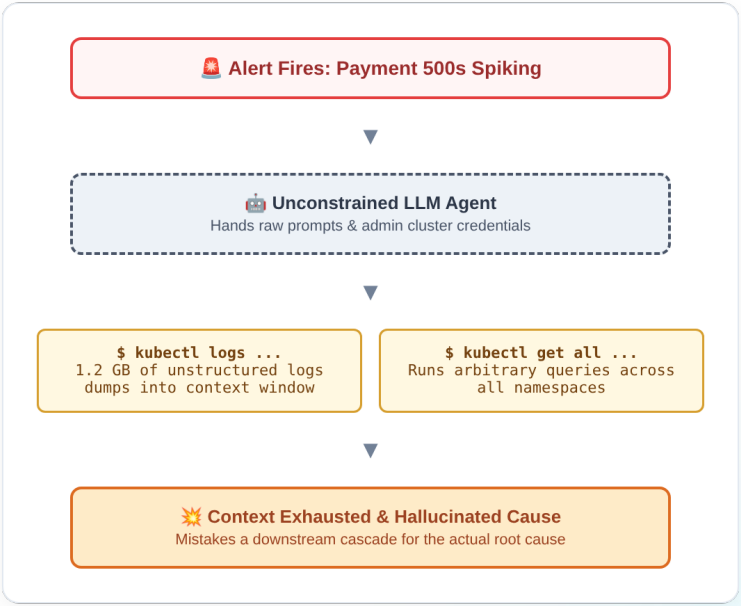
- What changed first in the distributed system
- Which service is the real root cause versus cascade victim
- How far the blast radius has spread across dependencies
- What exact hypothesis to verify next without guessing

The on-call reality:
You spend 40 minutes jumping across 12 dashboard tabs, grep-ing endless logs, and hoping your intuition holds.

The obvious instinct is to point an LLM at your logs

Why unconstrained diagnostic agents break down:

- **Massive context waste:** Dumping gigabytes of raw logs floods context windows with irrelevant noise.
- **Hallucinated causality:** Models mistake downstream cascade symptoms for primary root causes.
- **Unbounded latency & cost:** Open-ended exploration spins on endless log streams.
- **Security hazard:** Broad kubectl or cluster-admin privileges handed to model prompts.



Instead of searching blindly, bound the evidence first

Answer the first five minutes with three disciplined layers of proof:

1. Metrics (RED)

Rate, Errors, Duration & Saturation

Metrics establish correlation, not causation.

Their job is to isolate when the anomaly began and narrow candidate services to a small set.

2. Traces (Path)

Distributed Call Graph

Traces bound the path across services.

They identify which upstream caller triggered which failing downstream dependency in the call tree.

3. Logs (Targeted)

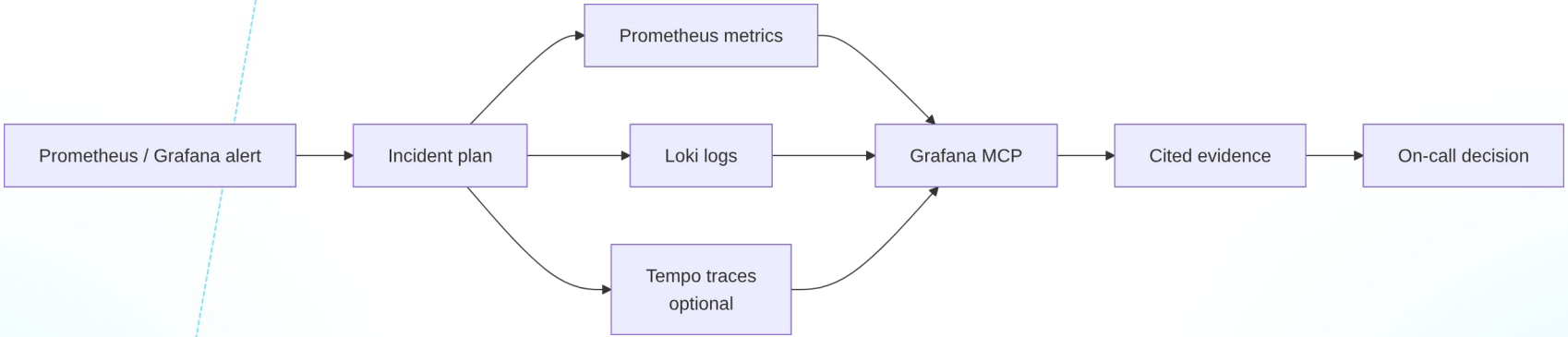
Exception & Stack Proof

Targeted error signature proof.

Queried ONLY after metrics and traces isolate the time window to seconds and target to one pod. Used for confirmation, never discovery.

Grafana is where telemetry turns into correlation

Prometheus, Loki, and Tempo hold the data. Grafana correlates them, and MCP exposes read-only questions.



- **Evidence Queries:** Client: internal telemetry / Repo: fixed templates over 3 services
- **Correlation Plane:** Client: enterprise Grafana / Repo: public starter dashboard
- **MCP Gateway:** Client: internal auth proxy / Repo: standalone read-only server
- **Incident State:** Client: full incident ticketing / Repo: clean persisted state machine
- **Validation:** Client: canary check suite / Repo: one-click fault toggle

Taking an enterprise pattern and making it reproducible

How we rebuilt proprietary production controls into an open demo

Never let an LLM write raw PromQL in production

The gateway translates intent into verified recording rules

- **Hallucination Risk:** Models hallucinate metric names and label keys under incident stress.
- **Cluster Safety:** Unguarded regex PromQL can easily crash production Prometheus instances.
- **Deterministic RED:** Vetted recording rules pre-compute RED metrics deterministically.
- **Parameter Injection:** The model supplies parameters; the gateway executes verified templates.

```promql
// Parameterized query execution:
get_service_health({
  service: "payment-service",
  time_range: "15m"
})

// Gateway executes pre-compiled recording rules:
sum(rate(http_requests_total{service="payment",status=~"5.."}[5m]))
/
sum(rate(http_requests_total{service="payment"}[5m]))
```

Enforce security in schemas, not in prompts

Tool schemas act as executable, hard-coded guardrails

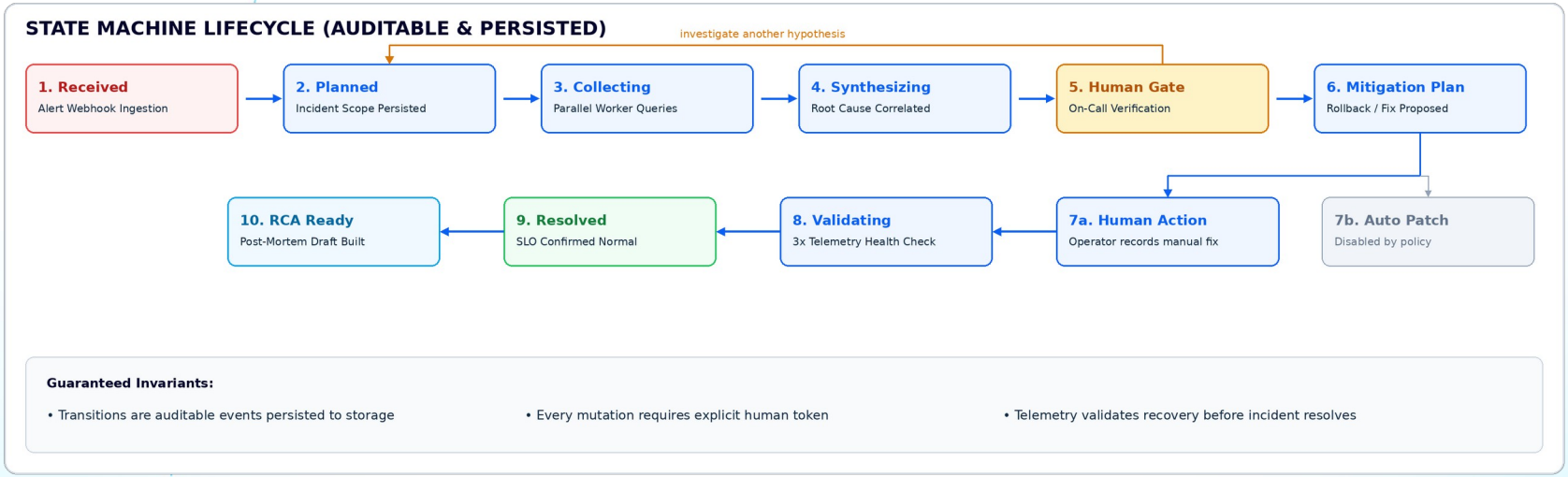
- **No Extra Properties:** additionalProperties: false rejects injected unexpected arguments.
- **Enum Constrained:** Enums constrain targets to verified service catalog entries only.
- **Bounded Windows:** Time windows are bounded: impossible to query full year history.
- **Immediate Rejection:** Violations reject instantly at MCP layer before touching datasources.

schema

```
{
  "name": "get_service_health",
  "description": "Read-only RED metrics for candidate service",
  "input_schema": {
    "type": "object",
    "properties": {
      "service": { "enum": ["web", "orders", "payment"] },
      "metric":  { "enum": ["errors", "latency", "saturation"] },
      "window":  { "enum": ["5m", "15m", "1h"] }
    },
    "required": ["service", "metric"],
    "additionalProperties": false
  }
}
```

Every state change must be an auditable human decision

An explicit state machine where transitions require proof, and automated actions remain locked.

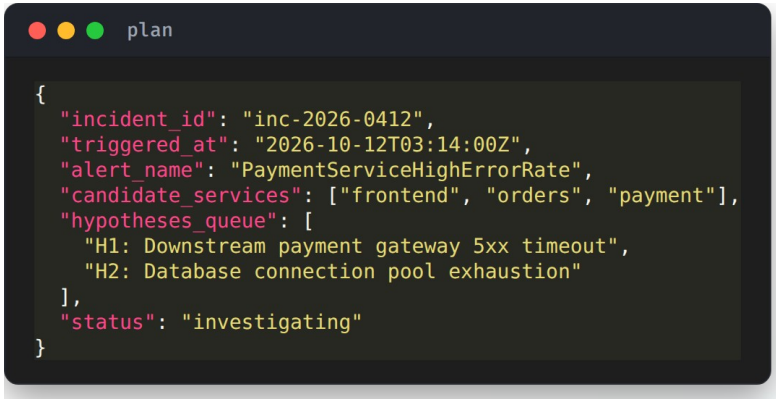


civo NUTANIX Red Hat DATADOG datacomm

The alert builds a shared plan before anyone runs queries

Persisted incident scope created immediately upon trigger

- **Persisted State:** Plan is persisted to storage before any worker executes diagnostic queries.
- **Live Collaboration:** Engineers can view and edit the hypothesis queue live in incident chat.
- **No Runaway Loops:** Prevents duplicate queries and runaway investigation branches.



Five focused workers asking five narrow questions

Parallel investigation without ambient cluster authority

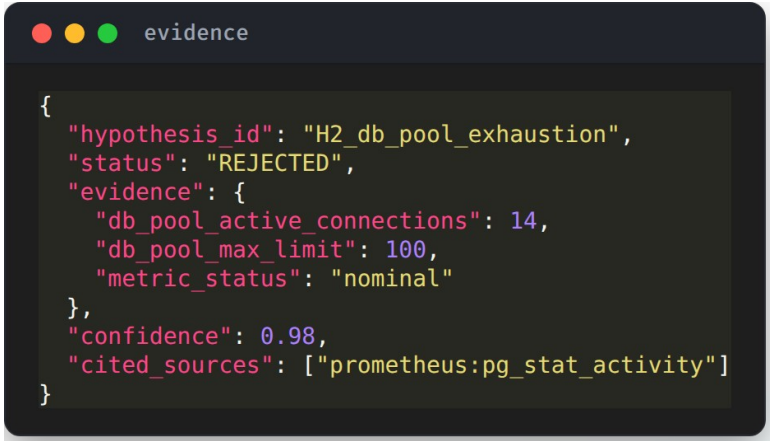
Worker Boundaries:

- **Metrics Worker:** Queries RED metrics & saturation. Zero PromQL.
- **Logs Worker:** Queries error signatures in a 3-minute window.
- **Trace Worker:** Inspects top 5 slowest/failing spans only.
- **Topology Worker:** Resolves service dependency tree from catalog.
- **Changes Worker:** Checks recent deployments & config diffs in past 30m.

Ruling out a false lead is just as valuable as the fix

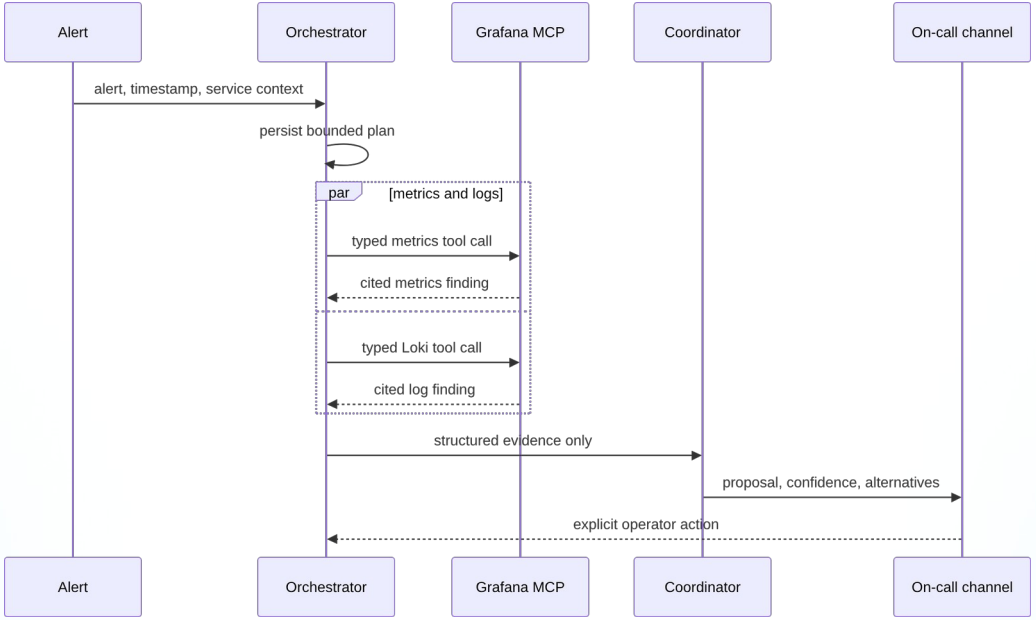
Structured evidence verdicts eliminate subjective debate

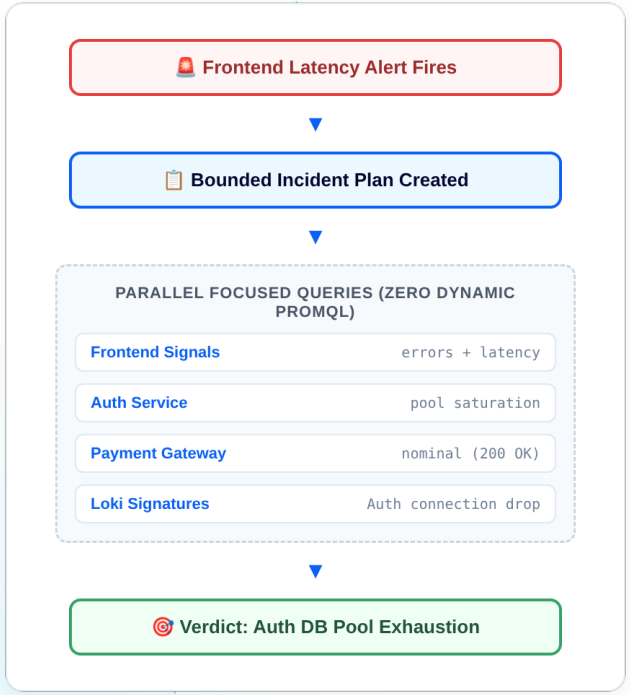
- **Equally Valuable:** A rejected hypothesis is as valuable as a confirmed one.
- **Clear Auditing:** Engineers immediately see what was ruled out and why.
- **No Circular Logic:** Prevents circular arguments and re-testing known good subsystems.
- **Restricted Context:** Coordinator receives structured verdicts, never raw dumps.



How the entire investigation unfolds end to end

Alert ➔ Incident plan ➔ Parallel workers ➔ Synthesized root cause ➔ Human gate





Architecture enforces trust, not polite prompts

Strict least-privilege security controls across four layers:

- **Orchestrator:** Schedules tasks & state. Has ZERO Grafana source credentials.
- **MCP Gateway:** Read-only access to Grafana. Rejects all mutations and write calls.
- **Workers:** Ephemeral processes with single-purpose schema tokens.
- **Mitigation Engine:** Requires explicit cryptographic token from human chat approval.

Chat buttons request actions, but the backend verifies

Slack and Teams ChatOps pattern with cryptographic nonces

- **No Direct Execution:** A chat button never triggers an action or shell script directly.
- **Server-Side RBAC:** The server validates actor identity, incident phase, and nonce freshness.
- **Replay Protection:** Stale or duplicate button clicks are rejected harmlessly.
- **Cryptographic Tokens:** Approved actions receive single-use scoped execution tokens.

chat.button

```
// Action button payload signed by chat bridge:
{
  "incident_id": "inc-0412",
  "action": "approve_rollback",
  "target_revision": "v2.4.1",
  "nonce": "req-98f7-exp30s",
  "signer": "chat-ops-bot"
}

// Backend verifies actor RBAC, phase & nonce freshness
```

A human applies the fix, but telemetry proves it worked

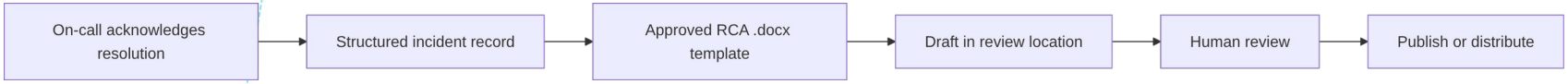
Closing the loop with automated verification cycles

Post-Mitigation Validation Cycle:

- **Mitigation Applied:** Engineer approves deployment rollback or traffic re-route.
- **Automated Health Check:** Gateway monitors 3 consecutive evaluation cycles (3x 60s).
- **Error Rate Re-check:** Verifies payment-service error rate drops below 0.1% SLO threshold.
- **Auto-Resolution:** Transitions state to Resolved only when telemetry proves recovery.

The post-mortem assembles itself while you fix the incident

The draft builds deterministically from the audit trail, waiting for human sign-off.



Automated Post-Mortem Assembly Lifecycle:

- **1. Resolution Acknowledged:** Incident recovery event freezes evidence collection and triggers post-mortem generation.
- **2. Structured Record Pull:** Gathers alert timestamps, hypotheses tested/rejected, and confirmed telemetry verdicts.
- **3. Template Merge:** Renders the data into the standardized incident report format without manual copy-pasting.
- **4. Draft Created:** Stores draft in designated incident review location with immutable audit links.
- **5. Human Review:** Engineer verifies accuracy, adds organizational context, and signs off.
- **6. Distribution:** Publishes final post-mortem report to incident channels and stakeholder teams.

Why bounding the evidence beats dumping the logs

Comparing the two architectures across five production dimensions

- **Query Scope:** Raw Log: unbounded dump / RCA Engine: bounded RED rules
- **Plan State:** Raw Log: implicit model thoughts / RCA Engine: persisted, auditable state
- **Token Cost:** Raw Log: 100k+ tokens / RCA Engine: < 4k structured tokens
- **Security Privileges:** Raw Log: kubectl cluster-admin / RCA Engine: read-only scoped MCP
- **Human Control:** Raw Log: hope model behaves / RCA Engine: state machine approval gates

Where this design deliberately stops

Clear boundaries that maintain engineering credibility

Hard Limits of the Engine:

- **Metrics give correlation, not causation:** Proving services degraded together != physical proof of bug.
- **Cannot fix missing telemetry:** If a service exports no RED metrics, the gateway cannot invent data.
- **Unmaintained dependency graphs fail:** Stale service catalogs cause the coordinator to miss cascade paths.
- **Not autonomous remediation:** Does not restart pods or alter routing without explicit human signoff.

How to build this incrementally without boiling the ocean

A 4-step adoption roadmap that delivers value from day one

Phased Adoption Roadmap:

- **Phase 1: Telemetry & Rules:** Standardize RED metrics and Prometheus recording rules on tier-1 services.
- **Phase 2: Grafana Correlation:** Build single-pane dashboard linking metrics, Loki logs, and Tempo traces.
- **Phase 3: Read-Only MCP Gateway:** Wrap dashboard queries in structured, parameterized tool schemas.
- **Phase 4: State Machine & RCA:** Add incident state machine, hypothesis queue, and chat approval gates.

Three principles to take back to your team

- **1. Put Grafana at the evidence plane:** Prometheus and Loki hold truth; Grafana correlates them. Build tooling at the correlation layer, not individual datasources.
- **2. The model never writes queries:** Tool schemas are executable policies. Expose narrow, parameterized questions with zero dynamic query syntax.
- **3. Persist the plan and the state:** Keep humans in control with auditable state transitions, visible hypotheses, and explicit approval gates.

Try the entire stack locally on your machine

Clone the repo and run the three-service demo with Docker Compose

Everything in this talk is runnable in the demo repo:

- Prometheus recording rules & alert configurations
- Grafana correlation dashboard with RED metric panels
- Read-only Model Context Protocol (MCP) gateway server
- Contained fault-injection switch to trigger synthetic incidents
- Step-by-step Makefile: make up && make fault

Repo: github.com/eltrovert/root-cause-analytics-engine

KCD
GROWING CLOUD NATIVE TOGETHER

THANK YOU

Sponsors:

civo

NUTANIX Red Hat DATADOG

datacomm

CO Organizer

Google Developer Group
Bandung

Partners:

Openinfra User Group Indonesia

Ardan labs

GDG Cloud Jakarta

KUBE EVENTS

LOKÉT

Google Developer Group
Depok